



Modelado de Arquitectura de Software

Diagramas de paquetes y componentes del modulo Lab2026V

Curso	Arquitectura de Software
Programa	Ingenieria de Sistemas
Institucion	Universidad de Antioquia
Estudiante	Cristian Echeverry
Modulo analizado	Backend Lab2026V - Spring Boot y pipeline CI/CD

Informe elaborado con base en la actividad de clase sobre descripcion de arquitectura general con UML, donde se solicita construir el diagrama de paquetes y el diagrama de componentes del modulo asignado, considerando tambien los componentes externos con los que interactua el sistema.

1. Introduccion

Este informe presenta el modelado arquitectonico del modulo Lab2026V mediante UML estructural. La idea central no es solamente mostrar cajas y flechas, sino explicar como se organiza el backend, que responsabilidades tiene cada parte y por que esa organizacion ayuda a mantener el sistema claro y facil de evolucionar.

La actividad se enfoca en dos vistas principales: el diagrama de paquetes, que permite observar la organizacion logica del codigo, y el diagrama de componentes, que muestra los bloques principales del sistema, sus interfaces y la relacion con herramientas externas como GitHub Actions, Docker, SonarCloud, JaCoCo y AWS ECR.

2. Objetivo del trabajo

El objetivo es construir y justificar dos diagramas UML para el modulo Lab2026V: un diagrama de paquetes y un diagrama de componentes. Ambos diagramas deben reflejar una arquitectura ordenada, basada en capas logicas, con alta cohesion y bajo acoplamiento.

- Representar la estructura logica del modulo mediante paquetes.
- Representar la estructura de alto nivel mediante componentes e interfaces.
- Mostrar la interaccion del backend con clientes, persistencia y herramientas externas.
- Relacionar el modelado con buenas practicas de arquitectura y CI/CD.

3. Fundamento arquitectonico

El modulo se interpreta como una aplicacion backend construida con Spring Boot. Para su descripcion se adopta una arquitectura por capas. Esta decision es coherente con el principio de separar responsabilidades: la presentacion no debe conocer los detalles de persistencia, la logica de negocio no debe quedar dispersa en los clientes, y el dominio debe mantenerse lo mas desacoplado posible de los detalles tecnicos.

Desde el punto de vista de disenio, se busca que cada paquete tenga una responsabilidad principal. Esta idea corresponde a la alta cohesion. Al mismo tiempo, las dependencias se organizan en una direccion controlada para evitar que todos los modulos dependan de todos. Esto reduce el acoplamiento y facilita pruebas, mantenimiento y reemplazo de componentes.

Capa / paquete	Responsabilidad principal	Justificacion
Frontend / UI	Cliente que consume la API REST.	Se mantiene separado del backend para evitar distribuir la logica de aplicacion en los clientes.
controller	Expone endpoints HTTP.	Actua como entrada al sistema y delega la logica al servicio.
service	Orquesta casos de uso.	Funciona como fachada entre la capa de presentacion y el dominio.
domain / dto	Modela datos y estructuras de intercambio.	Agrupar conceptos propios del modulo y evita mezclar reglas con infraestructura.
repository / persistence	Abstrae acceso a datos.	Evita que el dominio conozca la forma concreta de almacenamiento.

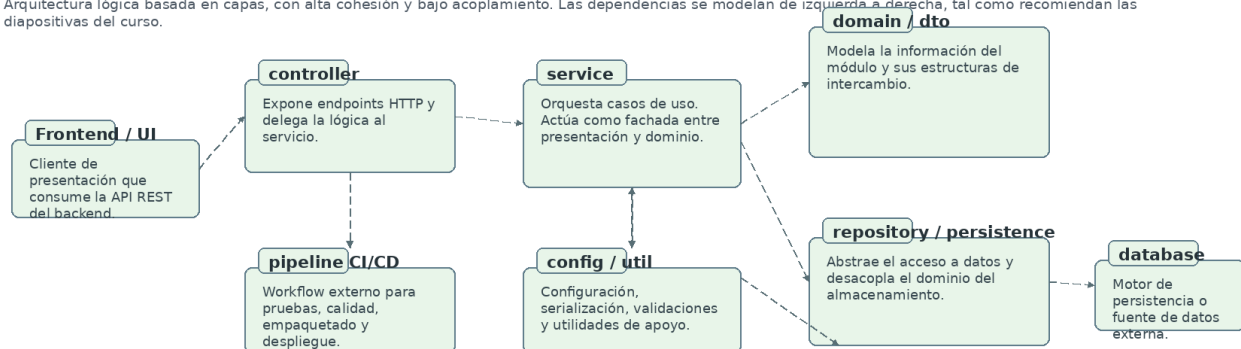
pipeline CI/CD	Pruebas, build, calidad y empaquetado.	Se modela como elemento externo porque soporta el ciclo de vida del artefacto.
----------------	--	--

4. Diagrama de paquetes

El diagrama de paquetes describe la organización lógica del sistema. En UML, un paquete actúa como un contenedor de elementos del modelo y permite agrupar responsabilidades relacionadas. Para este módulo, los paquetes se organizaron siguiendo el flujo natural de una aplicación web backend: el cliente entra por el controlador, el controlador delega al servicio, el servicio usa el dominio y, cuando se requiere persistencia, se comunica con el repositorio.

Diagrama de paquetes del módulo LAB2026V

Arquitectura lógica basada en capas, con alta cohesión y bajo acoplamiento. Las dependencias se modelan de izquierda a derecha, tal como recomiendan las diapositivas del curso.



Interpretación del diagrama

- controller depende de service; service centraliza los casos de uso y evita que la lógica de aplicación quede distribuida en los clientes.
- persistence desacopla el modelo del dominio de la tecnología de almacenamiento.
- config / util concentra elementos transversales.
- el pipeline CI/CD se modela como paquete externo porque interactúa con el artefacto pero no pertenece al núcleo del dominio.

Figura 1. Diagrama de paquetes del módulo Lab2026V.

El sentido de las dependencias se mantiene de izquierda a derecha para mejorar la lectura. Esta decisión también evita representar relaciones innecesarias. El paquete **controller** no accede directamente a la base de datos; su responsabilidad es recibir solicitudes y delegar. El paquete **service** centraliza los casos de uso, por lo que evita que la lógica de aplicación quede repetida o distribuida. El paquete **repository / persistence** desacopla el modelo del mecanismo de almacenamiento.

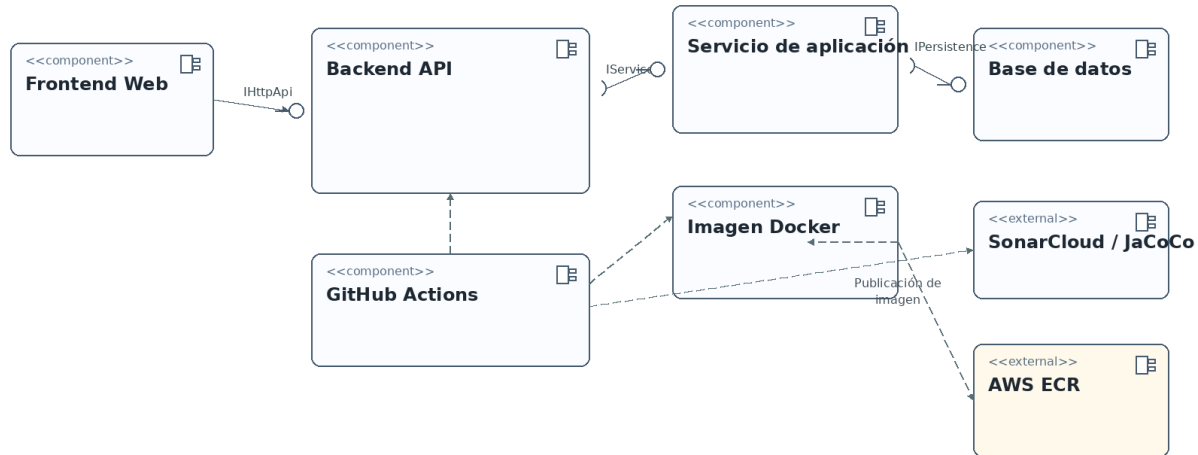
El paquete **pipeline CI/CD** no pertenece al núcleo funcional del sistema, pero se incluye porque el trabajo pide considerar otros componentes con los que el sistema interactúa. Este paquete representa el proceso externo de prueba, análisis de calidad, construcción, empaquetado y despliegue.

5. Diagrama de componentes

El diagrama de componentes permite describir el sistema desde una perspectiva de alto nivel. Aquí no se pretende mostrar todas las clases, sino los bloques principales que colaboran para que el módulo funcione. Cada componente encapsula una parte del sistema y se comunica con otros mediante interfaces.

Diagrama de componentes del módulo LAB2026V

Modelo estructural de alto nivel con componentes, interfaces proporcionadas y requeridas, y artefactos externos de soporte para pruebas, empaquetado y despliegue.



Interpretación del diagrama

• El componente Frontend consume la interfaz HTTP proporcionada por el Backend API. • El Backend API requiere un servicio de aplicación, y este a su vez depende de la persistencia. • GitHub Actions ejecuta pruebas, cobertura y empaquetado; luego construye la imagen Docker y puede publicarla en AWS ECR. • SonarCloud y JaCoCo se modelan como herramientas externas de calidad sobre el artefacto construido.

Figura 2. Diagrama de componentes del modulo Lab2026V.

El componente **Frontend Web** consume la interfaz HTTP expuesta por el **Backend API**. Este backend requiere un **Servicio de aplicación**, encargado de ejecutar los casos de uso. A su vez, el servicio requiere una interfaz de persistencia para comunicarse con la **Base de datos**. Esta separación permite que cada componente pueda evolucionar con menor impacto sobre los demás.

También se representan componentes externos de soporte al proceso de entrega: **GitHub Actions** ejecuta pruebas, cobertura y construcción; **SonarCloud / JaCoCo** apoyan la revisión de calidad; **Imagen Docker** empaqueta el artefacto; y **AWS ECR** actúa como registro para publicar imágenes. Con esto, el diagrama refleja tanto la estructura del software como su ciclo de vida técnico.

6. Relacion con CI/CD

El laboratorio tambien se relaciona con el proceso de integracion y despliegue continuo. En este contexto, el sistema no termina cuando el codigo compila: debe probarse, analizarse, empaquetarse y prepararse para ser desplegado. Por eso se incluyeron componentes asociados al pipeline.

Etapa	Herramienta	Proposito
Repositorio	Git / GitHub	Controlar cambios y centralizar el codigo.
Pruebas	JUnit 5	Validar que los endpoints y comportamientos esperados funcionen.
Cobertura	JaCoCo	Medir el porcentaje de codigo ejercitado por las pruebas.
Calidad	SonarCloud	Detectar defectos, vulnerabilidades y deuda tecnica.
Build	Maven	Compilar, ejecutar pruebas y generar el artefacto JAR.
Empaquetado	Docker	Crear una imagen portable y reproducible.
Registro	AWS ECR	Publicar la imagen para su posterior despliegue.

Este flujo permite reducir errores de integracion, acelerar la entrega del software y asegurar que cada cambio sea revisado antes de considerarse listo. En una entrega academica, el punto mas importante es demostrar que la arquitectura no solo se piensa para ejecutar localmente, sino tambien para ser probada y empaquetada de forma automatizada.

7. Cumplimiento de lo solicitado

La actividad solicita construir el diagrama de paquetes y el diagrama de componentes del modulo asignado, considerando otros componentes que el sistema necesite para interactuar o llamar. La propuesta cumple con esto de la siguiente manera:

- El diagrama de paquetes representa la organizacion logica del modulo mediante paquetes con responsabilidades separadas.
- El diagrama de componentes representa los bloques de alto nivel, sus interfaces y dependencias.
- Se agregan componentes externos relevantes: cliente frontend, base de datos, Docker, GitHub Actions, SonarCloud / JaCoCo y AWS ECR.
- La direccion de dependencias evita relaciones circulares y mantiene la lectura de izquierda a derecha.
- La propuesta mantiene alta cohesion por paquete y bajo acoplamiento entre capas.

8. Conclusiones

El modelado UML del modulo Lab2026V permite ver con claridad como se organiza el sistema antes de entrar a revisar codigo fuente. El diagrama de paquetes ayuda a entender la estructura logica, mientras que el diagrama de componentes permite observar los bloques principales que intervienen en la ejecucion y entrega del software.

La separacion por capas mejora la mantenibilidad porque cada parte del sistema tiene una responsabilidad concreta. Ademas, la inclusion de CI/CD, Docker y herramientas de calidad muestra que la arquitectura no se

limita al desarrollo local, sino que contempla procesos actuales de construccion, validacion y despliegue.

En conclusion, el trabajo representa una arquitectura ordenada y defendible para un modulo backend moderno. La propuesta puede llevarse a herramientas como Lucidchart o GenMyModel manteniendo las mismas relaciones, interfaces y componentes mostrados en los diagramas.

Referencias

- [1] D. Botia, "Descripcion de la arquitectura general con UML," material de clase, Universidad de Antioquia, 2023.
- [2] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley, 2020.
- [3] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.
- [4] GitHub Docs, "GitHub Actions documentation." Disponible: <https://docs.github.com/actions>.
- [5] Docker Inc., "Docker documentation." Disponible: <https://docs.docker.com>.
- [6] SonarSource, "SonarCloud documentation." Disponible: <https://docs.sonarcloud.io>.